COMMERCE ARCHITECTURE DECISION

Hydrogen or a Bespoke Xano + Cloudflare Stack

Hydrogen is a framework for one Shopify storefront. A bespoke stack is your own backend and data layer, sitting under several storefronts. **Compared as of Hydrogen 2026-10.**

For: **Teams choosing between Shopify Hydrogen on Oxygen and a bespoke Xano + Cloudflare commerce stack**

Updated: **2026-10-29 · Hydrogen: 2026-10 · Shopify CLI: 5.0.0**

Short answer

They aren't substitutes. Hydrogen replaces only the storefront layer. Pick it when a client wants a standard Shopify storefront with Shopify-managed hosting. Keep the bespoke stack as the core when data comes from more than Shopify, when one backend serves several brands or regions, or when consent and data residency must be enforced in code you control.

Headless only, or CMS-agnostic

Hydrogen and Next.js can only be headless with Shopify. They replace the Online Store theme and run beside Shopify, so a store that adopts them leaves the theme editor and its theme apps behind.

Webflow, Xano and Cloudflare are CMS-agnostic. The same worker, data layer and web components serve WordPress, Shopify, AEM and Next.js, Astro, Svelte or Vue front ends, and Webflow's CMS can itself act as a headless content

source. Each can run headless, or be brought into a Shopify native theme through a theme app extension, so the store keeps its theme editor and checkout.

App blocks need an Online Store 2.0 theme with JSON templates; app embeds work in vintage themes too. Shopify suggests keeping each block's own JavaScript under 10 KB compressed, so the block loads a small loader from the worker, as the CRM Sync Brand Kit extension does.

Request path

Hydrogen + Oxygen

1. **Browser**: Hydrated React app
2. **Oxygen worker**: React Router renders each page on the server
3. **Storefront API + Customer Account API**: Products, cart, checkout, customer accounts
4. **Shopify**: The only source of truth

Bespoke Xano + Cloudflare

1. **Browser**: Webflow or 11ty pages, enhanced with small scripts or web components
2. **Cloudflare Worker**: Consent gate, routing per tenant, auth, cart and checkout calls
3. **Xano**: Source of truth for product info (PIM), customers (CRM) and consent records
4. **Shopify, Webflow CMS, drop-ship suppliers**: Feed Xano through webhooks and sync jobs; assets live in R2

The Hydrogen CLI got smaller

In 2026-10 the Hydrogen CLI only handles the Shopify platform. You build, run and generate code with React Router's own tooling. Shopify CLI `@shopify/cli-hydrogen` 13.0.1 and later stops the old commands with an error.

Still in the Hydrogen CLI

- **`login`** / **`logout`**: Sign in to a shop
- **`link`** / **`unlink`** / **`list`**: Connect the project to a storefront
- **`env list`** / **`pull`** / **`push`**: Sync environment variables
- `customer-account-push`: Push Customer Account API config
- `deploy`: Build and deploy to Oxygen

2026-04 and earlier only

- **`dev`** / **`build`** / **`preview`**: Use the framework's commands
- `codegen`: GraphQL types from Storefront API queries
- `generate route(s)`: Route scaffolding
- **`setup`** / **`upgrade`** / **`check`**: CSS, markets, Vite setup and upgrades

Projects that run `npm run dev --codegen` are on 2026-04 or earlier and need Shopify's migration guide to move to 2026-10. `hydrogen init` still creates a 2026-04 project.

Comparison

CONCERN	HYDROGEN + OXYGEN	BESPOKE XANO + CLOUDFLARE	EDGE
Scope	One storefront: pages, cart, customer accounts	Backend, PIM, CRM, consent, webhooks, agent checkout, several front ends	Bespoke
Source of truth	Shopify	Xano; Shopify is one input	Depends
Rendering model	Headless only, like Next.js on Shopify; the Online Store theme is replaced	CMS-agnostic: WordPress, Shopify, AEM, Next.js, Astro, Svelte, Vue, either headless or inside a Shopify theme through app blocks and app embeds	Bespoke
Front end	React, rendered on the server	Any: Webflow, 11ty, UIkit with petite-vue, no React required	Bespoke
Hosting	Oxygen, run by Shopify, included with the plan	Workers, KV, R2 on your Cloudflare account	Hydrogen
Data residency	Oxygen decides; little regional control	You choose regions, including a Xano instance in the visitor's country	Bespoke
Consent	Customer Privacy API is built in, but every page renders on the server before the visitor's choice is known	A gate in the worker decides what leaves, per region and per tenant	Bespoke
Brands and regions	One Hydrogen app per storefront	One worker serves many tenants from config	Bespoke
Non-Shopify data	Possible, but you fetch and cache it yourself	Native: CMS, drop-ship feeds, asset stores	Bespoke
Cart and checkout	First-class, kept current by Shopify	You call the Storefront API and keep up with its versions	Hydrogen
Upkeep	Shopify handles deploys, scaling, API upgrades	You own token refresh, webhook queues, deploys, monitoring	Hydrogen
Time to first launch	Fast for a standard storefront	Slower to stand up; reused across projects afterwards	Hydrogen

CONCERN	HYDROGEN + OXYGEN	BESPOKE XANO + CLOUDFLARE	EDGE
SAP, Salesforce, Merchant Center	Attach to Shopify through apps and middleware; the storefront isn't involved	Attach to Xano and the worker, behind the same consent gate	Depends
Sign-in	Shopify customer accounts (OIDC) built in, for one store	One `/auth/me` across Google, Shopify and Kakao identities and every tenant	Depends
Agent checkout (UCP)	Shopify publishes the store's UCP profile and runs checkout; AP2 mandates aren't part of its documented flow	You verify AP2 mandates and set limits per agent	Depends
Kakao Pay, Samsung Pay	Only through a Shopify-approved payments app	Direct PG or Stripe integration in the worker	Bespoke
Native apps	A separate iOS or Android app using Checkout Kit and the Storefront API; no desktop shell	Tauri shells wrap the same web app, with one Rust core shared by native and web	Bespoke
Wallet passes	Through loyalty apps from the App Store	Google Wallet, Samsung Wallet and KakaoTalk passes built from your own ledger	Bespoke
App and game distribution	Separate native app per store; no game path	One PWA across web, stores, Galaxy Store and games, with Shopify as the security layer	Bespoke
3D and STL printing	GLB and USDZ product media, public on Shopify's CDN; no STL	PIM Sync for 3D files, three.js customizer, STL export, domain-locked loading	Bespoke
Permissions	Access scopes per app and resource	Caps per peer and direction, from `/auth/me`, enforced in the worker and an Axum service	Bespoke
Skills needed	React, React Router, GraphQL	Workers, XanoScript, GraphQL, the CMS of choice	Even

Integrating SAP, Google Merchant Center and Salesforce

With Hydrogen, these systems connect to Shopify, not to the storefront. Hydrogen only shows whatever has already reached Shopify. In the bespoke stack they connect to Xano and the worker, so the same consent and residency rules cover them.

SAP

S/4HANA, ECC, Business One · **Edge:** Bespoke

HYDROGEN + OXYGEN

Connects through. Middleware between the Shopify Admin API and SAP: SAP Integration Suite, or an iPaaS such as Boomi or Celigo.

What syncs. Material master to products, stock levels to inventory, orders and fulfilment back to SAP.

Watch for. Shopify becomes a second product master. SAP fields with no Shopify equivalent end up in metafields.

BESPOKE XANO + CLOUDFLARE

Connects through. The worker or Xano calls S/4HANA OData APIs. Integration Suite relays IDocs, and S/4HANA business events arrive through Event Mesh, both posting to a worker endpoint and a Cloudflare Queue.

What syncs. SAP stays master for material, price and stock. Xano adds copy, media and translations, keyed by material number, and pushes the result to Shopify.

Watch for. You own retries, field mapping and SAP authentication (OAuth or client certificates).

Google Merchant Center

Shopping ads, free listings, agentic checkout · **Edge:** Depends

HYDROGEN + OXYGEN

Connects through. Shopify's Google & YouTube app, which syncs the Shopify catalog to Merchant Center.

What syncs. Products, prices and availability, with a feed for each Shopify Market.

Watch for. Product links must point to the Hydrogen domain, and product pages need structured data that matches the feed, or items are disapproved. Titles and attributes can only be adjusted through metafield mapping.

BESPOKE XANO + CLOUDFLARE

Connects through. Merchant API, which replaces the Content API for Shopping. Xano or a scheduled worker inserts products for each feed label and language, or Merchant Center fetches a scheduled feed file from R2.

What syncs. Price, sale price and sale dates for each market, so prior-price rules such as Omnibus match what Google shows.

Watch for. Account and data source setup, API quotas, and keeping the landing-page price identical to the feed price.

Salesforce

Sales, Service and Marketing Cloud, Data 360 · **Edge:** Bespoke

HYDROGEN + OXYGEN

Connects through. A Shopify-to-Salesforce connector app or MuleSoft, with Shopify Flow and webhooks for events. MuleSoft's Shopify connector is built on Shopify's REST Admin API, which Shopify made legacy on 1 October 2024. Since 1 April 2025 new public apps must use the GraphQL Admin API, and Shopify began switching off REST endpoints in October 2025. Products with

more than 100 variants (the limit became 2,048 on 15 October 2025), metaobjects and newer Markets features are GraphQL-only, so the connector can't reach them. Use MuleSoft's HTTP connector with the GraphQL Admin API for new work.

What syncs. Customers, orders and email and SMS marketing consent. Customers sign in through the Customer Account API, so Salesforce sees them only after Shopify does.

Watch for. Shopify records consent per channel. It has no field for consent by purpose or by region.

BESPOKE XANO + CLOUDFLARE

Connects through. Salesforce REST and Bulk API 2.0 from the worker, through a connected app using the OAuth client-credentials flow. Changes come back as Flow HTTP callouts to a worker endpoint. The Data 360 (formerly Data Cloud) Ingestion API takes behaviour events.

What syncs. Xano customer and consent records map to Salesforce's consent objects (Individual, Contact Point Type Consent).

Watch for. The Pub/Sub API needs gRPC, which Workers can't run as a client, so use Flow callouts or CometD streaming for change events.

What GraphQL and Merchant API require of CRM and ERP data

Shopify's GraphQL Admin API and Google's Merchant API each fix the shape a product, price, customer or order must arrive in. That shape flows back as requirements on SAP, Salesforce and the PIM. Both stacks face the same requirements; the difference is where the mapping lives. With Hydrogen it sits in

connector apps or middleware. In the bespoke stack it sits in Xano, once, and feeds every peer.

The deprecation calendar

- **1 Oct 2024** (Shopify): REST Admin API becomes legacy.
- **1 Apr 2025** (Shopify): New public apps must use the GraphQL Admin API only.
- **Jul 2025** (Google): Merchant API v1 is generally available as the successor to the Content API for Shopping.
- **Oct 2025** (Shopify): REST endpoints start switching off in waves, beginning with product counts.
- **15 Oct 2025** (Shopify): Products can have 2,048 variants; REST-based tools break above 100.
- **1 Jan 2026** (Shopify): No new legacy custom apps in the Shopify admin. New custom apps are made in the Dev Dashboard and get access tokens through the client-credentials grant that expire after 24 hours, so there's no permanent token to copy into a connector.
- **28 Feb 2026** (Google): Merchant API v1beta sub-APIs are discontinued.
- **18 Aug 2026** (Google): Content API for Shopping is retired. Feeds must write through Merchant API.
- **1 Jan 2027** (Shopify): Public apps that call the Admin API must use expiring offline access tokens. Apps that haven't migrated are delisted and blocked from new installs. Custom apps, merchant-created apps and online tokens are exempt.
- **15 Jan 2027** (Shopify): Merchants see a warning in the admin for apps still on non-expiring tokens.
- **1 Apr 2027** (Shopify): Admin API requests with non-expiring offline tokens return HTTP 403.

- **Every quarter** (Both): Shopify ships a new API version, each supported for about a year. Google gives 12 months' notice before retiring a Merchant API version.

Why SAP S/4HANA and the Merchant feed feel it first

A Google feed reads the catalogue from Shopify and writes it to Merchant Center, so it sits on both migrations at once: Shopify REST to GraphQL on the read side, Content API to Merchant API on the write side. A feed can't push a catalogue it can no longer read. SAP integrations built on REST connectors hit the same wall, and SAP's OData shape (materials, plants, condition records) matches neither GraphQL nor Merchant API.

Xano as the conversion layer

Xano keeps one canonical record per product, price, customer and order. Adapters translate in each direction: SAP OData into the canonical record, the canonical record out to Shopify GraphQL bulk mutations and to Merchant API inputs in micros per feed label. Older systems and MuleSoft flows keep calling Xano's REST endpoints unchanged while Xano speaks GraphQL to Shopify. When Shopify or Google ships a breaking version, one adapter is amended instead of every connected system. Token refresh moves into the same layer: Xano and the worker renew expiring Shopify tokens per store, so SAP, MuleSoft and feed tools never hold a Shopify token that can lapse.

DATA	SHOPIFY GRAPHQL	GOOGLE MERCHANT API	WHAT CRM AI
Product identity	Global IDs such as `gid://shopify/ProductVariant/…`	One product per `contentLanguage~feedLabel~offerId`, for example `en~US~SKU123`	One stable SI crosswalk fro number to Sh offer ID and S
Variants	Options and up to 2,048 variants per product; REST tools break above 100	One item per variant, grouped by item group ID	Option values (size, colour, the title
Price	Decimal amount and currency; prices per market through Markets catalogues	Integer `amountMicros` and `currencyCode`, sale price with start and end dates, per feed label	A price per m sale windows prior price ea need, such as
Identifiers	Barcode field on the variant	GTIN, brand and MPN, or a declaration that none exist	GTIN, brand a part number typed in at lis
Language	Translations per locale through the Translations API	Content language fixed per product or data source	Translated tit per language truth for each
Stock	Inventory quantity per location	Availability as a fixed value (in stock, out of stock, preorder), optionally per region	Stock by war the countries serves
Custom data	Metafields and metaobjects, GraphQL only	Defined product attributes	Typed attribu not free-text
Customers and consent	Customer ID with email and SMS marketing consent: state, opt-in level and the time it changed	No customer data	Consent per timestamp ar in step with S objects
Orders and money	Each amount in both shop currency and the currency the customer paid in	No order data	ERP postings currencies ar used
Sync mechanics	Cursor pagination, bulk exports as JSONL files, rate limits by query cost, webhooks	Product inserts processed after a delay of minutes; batching by parallel requests	Queue-based resumable jo page REST lo

Upstream blockers when the ERP feed is REST-shaped

- Products over 100 variants, metaobjects and newer Markets data can't be written or read through REST, so a REST connector silently drops them.
- Merchant API rejects or disapproves items without GTIN, brand and a price in the feed's currency, so missing ERP fields surface as ad and listing losses.
- Prices sent as decimals must become integer micros for Google and stay decimals for Shopify; doing that conversion in two places lets them drift.
- Consent without a timestamp or opt-in level can't be mapped to Shopify or to Salesforce's consent objects, and must not be back-filled.

Sign-in, UCP mandates and Korean wallets

Hydrogen handles these through Shopify, so what it can do is whatever Shopify offers. In the bespoke stack, the worker connects each provider directly and stores the result in Xano under one user.

Sign-in and Auth Me

Google, Shopify customer accounts (OIDC), Kakao Login · **Edge:** Depends

HYDROGEN + OXYGEN

Connects through. The Customer Account API, an OAuth 2.0 and OpenID Connect flow with PKCE. Hydrogen's `customerAccount` helper handles login, callback and logout.

Google and other providers. Limited to the sign-in options Shopify customer accounts offer. Shopify Plus stores can connect an external identity provider; Multipass works only with legacy accounts.

Watch for. The session belongs to one store. A second brand or region means a second login.

BESPOKE XANO + CLOUDFLARE

Connects through. The worker runs each OIDC flow: Google Identity Services, the Shopify Customer Account API (discovered from the shop's `/.well-known/openid-configuration`) and Kakao Login. It checks each ID token against the provider's keys, then Xano issues its own token.

What Auth Me returns. One user with the claims from each linked identity, the extras stored for that user, and their capability caps, the same for every tenant.

Watch for. Link accounts only on a verified email or an explicit link step, never on an unverified email match.

```
GET /auth/me
{
  "logged_in": true,
  "user": { "id": 4182, "email": "…" },
  "claims": { "google_sub": "…", "shopify_customer_gid": "…" },
  "extras": { "region": "KR" },
  "entitlement": { "caps": ["checkout", "agent:delegate", "syndicate:out:webflow"] }
}
```

Example response; field names are illustrative.

Agent checkout with UCP

Universal Commerce Protocol with AP2 mandates · **Edge:** Depends

HYDROGEN + OXYGEN

Connects through. Shopify publishes the store's UCP profile at `/.well-known/ucp` and runs checkout, so agents can buy from it. The merchant stays the merchant of record. Hydrogen isn't in that path.

Mandates. AP2 mandates are an optional UCP extension that the business verifies. Shopify's agent checkout docs don't cover them.

Watch for. You work within the checkout Shopify exposes. Custom limits per agent or per customer aren't part of it, and agents only see the Shopify catalog.

BESPOKE XANO + CLOUDFLARE

Connects through. The worker publishes the UCP business profile at `/.well-known/ucp`, advertises the AP2 mandates extension, checks each mandate's signature, then creates the Shopify checkout or order.

Mandates. Stored in Xano against the user and the agent. The intent mandate's limits (amount, merchant, expiry) become capability caps the worker enforces.

Watch for. You maintain the spec version, key rotation and an audit trail that can show what each agent was allowed to do.

Kakao Pay and Samsung Pay

Korean wallets for a Korea-to-US storefront · **Edge:** Bespoke

HYDROGEN + OXYGEN

Connects through. Shopify Checkout only, so Hydrogen can't add a payment method of its own. A Korean wallet has to come from a payments app. The Shopify App Store lists Toss Payments and a third-party KakaoPay Checkout app; confirm each one runs inside checkout rather than redirecting off-site.

What syncs. The order and payment stay inside Shopify; the provider settles with the merchant.

Watch for. Shopify Payments isn't offered in Korea, and Samsung Pay isn't a built-in wallet the way Apple Pay, Google Pay and Shop Pay are.

BESPOKE XANO + CLOUDFLARE

Connects through. Kakao Pay's own API from the worker, with secret-key auth: a ready call returns the payment ID and redirects for app, mobile and desktop,

then an approve call confirms it with the returned `pg_token`. Alternatively, a Korean PG such as KG Inicis or NHN KCP, which covers both Kakao Pay and Samsung Pay, or Stripe's Korean payment methods.

What syncs. After payment, the worker creates the Shopify order with the transaction marked as paid through an external gateway, so fulfilment and reporting stay in Shopify.

Watch for. Stripe's Korean methods charge in KRW, to customers in Korea, and need a merchant based in the US, UK, Japan, Hong Kong, Singapore or the EEA, not Korea. PIPA restricts sending Korean personal data abroad, so keeping it in a Seoul-region Xano instance is the simpler policy. Test the app-to-app redirect on mobile.

Apps, games, wallets, 3D and permissions

The bespoke stack ships the same web front end as a website, an installable app, a desktop app and a TV app, with a shared Rust core inside each, and Shopify stays the security layer behind all of them. Its wallet support covers two separate jobs: paying with a wallet and holding a pass in one. 3D models and print files travel through PIM Sync, and every permission is scoped by the direction the data moves. Reference builds: the Ondol Life site and thermostat app, and the fo1 Korea storefront, which runs Kakao Pay checkout in test mode.

- Ondol floor app: Thermostat app: two heating zones read from the floor probes
- Mobile thermostat on the site: The same app presented on the Ondol Life website
- Ondol estimate and deposit: The web app the Tauri planner loads: estimate, consultation request, Kakao Pay test deposit

- fo1 Korea storefront: Kakao Pay ready and approve, then one Shopify draft order per vendor
- fo1 client and architect portal: Projects, invoices and payments for clients and architects
- CRM Sync desktop: Tauri desktop app around the CRM Sync configurator, with a Mac download
- fo1 3D customizer: Configure a chair, sofa or sectional in 3D, in any swatch, and send the configuration with a quote
- PIM Sync 3D: 3D models and print files synced from Xano and R2 to Shopify and Webflow
- Secure STL print-file viewer: Sandboxed viewer that loads print files from R2 on its own origin

Native app capability

Desktop, mobile and TV from one front end · **Edge:** Bespoke

HYDROGEN + OXYGEN

Connects through. A separate native app. Shopify's Checkout Kit shows Shopify Checkout inside an iOS, Android or React Native app, and the app reads products from the Storefront API.

Shared code. None with the Hydrogen site. Logic such as pricing rules or calculators is rewritten in the app, or called from a server.

Watch for. No desktop or TV shell, and two codebases to keep in step.

BESPOKE XANO + CLOUDFLARE

Connects through. Tauri 2 wraps the existing web app in a Rust host with the system webview. The same front end also ships as an installable web app and as a Samsung or LG TV app.

Shared code. A Rust core crate holds the calculations. Tauri links it natively, and the website loads the same crate compiled to WebAssembly, so the app and the page can't give different answers.

Capabilities. Tauri denies native access unless a capability file grants it, per window. When the app loads remote content, list that domain and grant it only what it needs, for example opening links in the system browser.

Watch for. App store signing and review, and pinning the WebAssembly tooling to the exact version the core was built with. Keep commerce out of safety logic: a purchase can unlock a feature, never change a device limit.

Hybrid wallet capability

Paying with a wallet, and holding a pass in one · **Edge:** Bespoke

The same brands sit on both sides, with different APIs: Google Pay pays and Google Wallet holds; Samsung Pay pays and Samsung Wallet holds; Kakao Pay pays and a KakaoTalk membership holds.

HYDROGEN + OXYGEN

Paying. The wallets Shopify Checkout offers: Shop Pay, Apple Pay and Google Pay. Other wallets need a payments app.

Holding passes. A loyalty app from the App Store issues the cards, from that app's own points data.

Watch for. Points live in the loyalty app, so a pass can't be checked against your own records.

BESPOKE XANO + CLOUDFLARE

Paying. Two kinds of rail. Agent rails (UCP, MPP, Kakao Pay) need an AP2 mandate. Wallets a person approves at the payment sheet (Google Pay, Apple Pay, Samsung Pay) need no mandate. A per-market settlement gate picks cross-border Stripe or a domestic PG, and blocks a rail a market doesn't allow.

Holding passes. One programme in Xano, published as a Google Wallet loyalty card, a Samsung Wallet card or a KakaoTalk membership. Balances fold from an append-only points ledger. Warranty passes work the same way from a service ledger.

Keeping passes honest. Each card records the ledger position it was built from. A card is out of date when the ledger has moved past that position, even if the balance looks the same, because an equal earn and spend leave the number unchanged.

Watch for. Each wallet provider has its own issuer approval, class rules and review. Warranty passes also go out of date as time passes, with no ledger event.

App and game distribution

PWA, app stores, Galaxy Store, Roblox and Unity, with Shopify as the security layer · **Edge:** Bespoke

HYDROGEN + OXYGEN

Distribution. The Hydrogen site can be made installable by hand. App store presence means a separate native app with Checkout Kit. Games have no path except calling the Storefront API from the game engine.

Security layer. Shopify handles sign-in, checkout and card data, for that one storefront.

Watch for. Each new surface is a new build with its own login.

BESPOKE XANO + CLOUDFLARE

Distribution. One progressive web app, shared by URL, listed in Galaxy Store as a PWA, and wrapped for the Apple and Google stores with Despia or Tauri. Roblox and Unity games link the player ID to the same Xano user.

Security layer. Shopify stays underneath every surface: customer accounts for identity, Shopify Checkout and tokenized wallets for payment, so no app or game ever handles card data. The worker checks caps and mandates before anything reaches Shopify.

Watch for. Digital goods sold inside an app store or game fall under that platform's billing; physical goods can go through web checkout. Game platforms add an age gate and children's privacy rules.

3D customizer, PIM 3D and STL printing

Configure in 3D, sync models and print files, load them securely ·
Edge: Bespoke

HYDROGEN + OXYGEN

3D on the product page. Shopify product media accepts 3D models as GLB or USDZ, shown with Hydrogen's `ModelViewer` and in AR on phones. STL isn't accepted as product media.

Customizer. A third-party app draws the configurator; the chosen options ride on the order as line item properties.

Watch for. Files on Shopify's CDN are public to anyone with the link, and print files need storage outside product media.

BESPOKE XANO + CLOUDFLARE

3D PIM Sync. Xano holds each product's 3D files (GLB, USDZ, STL, OBJ) with a mirrored copy in R2. PIM Sync pushes GLB and USDZ to Shopify's 3D media, and model and print-file links to Webflow CMS.

Customizer. A three.js configurator rebuilds the model from the chosen pieces, sizes and fabric, and sends that configuration with the quote. A swatch photo the customer uploads stays in their browser.

STL printing. The same scene exports to STL with three.js, for scale models, sample parts and hardware. Use 3MF when the printer needs units or colour. Mesh checks (watertight, wall thickness, scale) can sit in the shared Rust core, so the website and the app reject the same files.

Secure model loading. R2 only serves allowlisted storefront domains, its bucket can't be listed, and Cloudflare's firewall applies. The STL viewer runs in a sandboxed iframe on its own origin and loads files from R2 there, so the host page never fetches them. Uploads are signed on the server.

Watch for. STL carries no units or colour, so fix millimetres in the export. A copy pushed to Shopify becomes public.

Permissions by direction

Inbound and outbound syndication with peers, on Tokio and Axum, from Auth Me claims and extras · **Edge:** Bespoke

HYDROGEN + OXYGEN

Permissions. Each app gets Shopify access scopes per resource (read or write products, orders, customers). Channels that syndicate products to peers each install with their own scopes.

Watch for. Scopes say what an app may touch, not which way data may move. A channel with product write access can also overwrite what came in from elsewhere.

BESPOKE XANO + CLOUDFLARE

Direction. Every flow is inbound (consent, customers, orders coming in) or outbound (products, prices and media syndicated to peers such as Shopify, Webflow, Merchant Center and SAP). A cap grants one direction for one peer; outbound product rights never let a peer write consent or customers back.

Auth Me claims and extras. `/auth/me` returns the user, the claims from their linked Google and Shopify identities, the extras stored for them, and their

entitlement caps. The worker gates on the caps; the page only uses them to hide controls.

Tokio and Axum. A Rust service on Tokio and Axum fronts Xano, SAP and Dataverse. It forwards only listed routes and methods, never caches signed-in requests, never forwards DELETE, and keeps SAP and Dataverse server-only behind a key that browsers can't send. It runs in Cloudflare Containers.

Watch for. Keep event records append-only and direction rules one-way: a suppressed contact stays suppressed, and consent is never back-filled.

```
Route list for the Axum service
GET  products/*        # outbound: peers read the catalogue
POST consent           # inbound: records a choice
PATCH sap:A_Product/*  # outbound to SAP, server-only
```

Example entries; route names are illustrative.

Where the payment gate runs: Worker or Axum

Who may pay with Kakao Pay is application code, not the runtime

Neither runtime decides who may pay with what. Tokio schedules tasks across a pool of threads, and Rust's ownership rules are checked when the code compiles. A Cloudflare Worker runs each request in its own V8 isolate. Both keep one request's memory away from another's, but a rule such as "Korean market plus the `settle_sovereign_kr` entitlement unlocks Kakao Pay" is code you write. It should gate on the market the purchase settles in and an entitlement cap, never on nationality.

STEP	WHERE	WHAT IT DOES
1. Verify the caller	Worker	Checks the token on the server and loads the caller's caps
2. Look up the market	Xano settlement policy and country gateway table	Returns the market's gateway, whether it is sovereign, and the cap it requires
3. Decide	Worker settlement gate	Refuses blocked, preview and sanctioned markets; returns 403 when a sovereign market's cap is missing
4. Settle	Worker, calling Kakao Pay	Runs ready and approve only after step 3 passes

Why the Worker. Every request already passes through it, so the rule sits beside the consent, tenant and mandate checks it depends on, and a refusal happens before anything reaches Xano, Shopify or Kakao. The request's `cf.country` is a useful signal, but it is where the IP address appears to be, not the market of settlement.

Where Axum fits. A second check behind the Worker for server-only routes such as SAP and Dataverse writes. If it ever takes payment routes, it verifies the token and caps itself instead of trusting the Worker's result.

Rust at the edge. The gate doesn't need Axum to be written in Rust: `workers-rs` compiles Rust to WebAssembly and runs it as a Worker, the same pattern as the shared Rust core in the Tauri apps.

Isolation by region. Keeping Korean personal data in Korea is a deployment choice, a Seoul-region service beside a Seoul Xano instance, not something a thread scheduler or memory manager does.

```
// Axum middleware: fail closed unless the market and cap match
let allowed = caller.market == "KR"
    && caller.caps.iter().any(|c| c == "settle_sovereign_kr");
if !allowed { return Err(StatusCode::FORBIDDEN); }
```

Certificates, PKI tokens and the data release

The security layer under edge-served data and privacy cookies, after the eMudhra CA compromise

Certificates prove who is on each end of a connection. The release gate decides whether data may leave, to whom and when. TLS proves the connection, not the bytes, so a valid certificate never unlocks data by itself.

STEP	CERTIFICATE'S ROLE	WHAT DECIDES
0. Inbound connection	The TLS server certificate proves the visitor reached the real domain. CAA DNS records limit which CAs may issue it.	Nothing is released
1. Who is calling	People present tokens. Machines such as a POS or SAP can present a client certificate (mutual TLS), which proves which machine; the Worker maps it to caps.	Identity only
2. Agent mandates	AP2 mandates are signed credentials checked against the issuer's published keys.	The caps and limits inside the mandate
3. Release decision	None. The Worker gate and the Xano release gate check consent at the system of record, market, caps and residency, fail closed and write the release ledger.	This is the release
4. Outbound call	Certificates prove the destination: public TLS to Google and Shopify, a client certificate to SAP, Samsung Pay's encrypted token, Apple Pay's merchant certificate at Stripe.	Delivery of data already approved
5. Audit	The ledger records which key or certificate fingerprint signed or carried each release.	Evidence, not a decision

WHAT IS A PKI TOKEN

A PKI token is a credential whose authority comes from a private key, checked against the matching public key or certificate. It has two common forms. A **hardware PKI token** is a USB key or smart card that holds a private key and its certificate; the key never leaves the device, which is how India's digital signature certificates are issued. A **software PKI token** is a signed token (JWS, or an

encrypted JWE) that anyone can verify against a published key set without calling the issuer.

Permissions it carries. Audience, scope or caps, market, and the subject it acts for, all inside the signature, so a holder can't widen them. The release gate still checks them against the system of record.

History restrictions. Limits in time and use: not-before and expiry; a unique ID so each token or mandate is used once or a set number of times, recorded in the ledger; revocation (a denylist for tokens, CRL or OCSP for certificates); and a read window, so a token reaches only records created after the subject's consent and inside the retention period, never back-filled history.

Key rotation. Signing keys move from next to active to retired. Tokens and certificates signed by a retired key still verify until they expire, and a compromised key is revoked instead of retired.

THE EMUDHRA GAP

In October 2026 a researcher showed that the private keys of eight eMudhra intermediate CAs could be recovered from their public certificates. Browsers blocked them within days, and Chrome removed the emSign ECC Root CA - G3 from its root store on 27 October 2026. Every site and client that trusted that chain had to replace certificates or patch trust stores, including Node, Python's certifi and Go. The lesson for edge-served data: TLS trust is rented from public CAs, and a CA failure breaks the layer that Secure cookies and edge delivery depend on.

COOKIE FLAG	WHAT IT STOPS	DEPENDS ON THE CA CHAIN
`Secure`	Sending the cookie over plain HTTP	Yes: a forged certificate lets an interceptor read it
`HttpOnly`	Script on the page reading the cookie	No
`SameSite`	Sending the cookie on cross-site requests	No
`__Host-` prefix	Subdomains setting or overriding the cookie; requires `Secure`, no `Domain` and `Path=/`	Yes
`Partitioned`	A third-party cookie following the visitor across sites	No

What closes the gap. CAA records with an incident contact, so only chosen CAs can issue; Certificate Transparency monitoring for unexpected certificates; patched trust stores in every server-side client, not only browsers; and consent kept as a signed server-side record, so a stolen or forged cookie can't rewrite what a visitor agreed to.

Bring your own CA. Device and code-signing certificates from a CA the business operates or chooses, with only the signing request leaving its network.

A visitor's first page load

The difference shows up before the visitor has said yes or no. Steps marked in pink are where data can leave without a decision; steps marked in blue are where the stack checks one.

Hydrogen + Oxygen

1. The request reaches Oxygen with its headers, IP and cookies.
2. The server renders the page and calls any APIs the route needs, including third-party ones.

3. HTML and the React bundle reach the browser.
4. The consent banner appears and the Customer Privacy API records the choice.
5. Analytics respect the choice from here on. Anything the server sent in step 2 already happened.

Bespoke Xano + Cloudflare

1. The browser loads static HTML from the edge; no personal data is used to build it.
2. The consent banner records the choice for the visitor's region.
3. The worker's gate checks that choice before forwarding anything to Xano, Shopify or analytics.
4. Data goes only to regions the tenant allows.
5. Scripts that personalise the page load after consent.

Hydrogen can be made consent-safe, but that is an audit you repeat for every route. In the bespoke stack the gate is in one place and applies to every route.

When to pick which

Choose Hydrogen when

- The client sells only through Shopify and wants one custom storefront.
- The team already works in React.
- Nobody on the client side will run infrastructure.
- Shopify-managed hosting and cart updates matter more than control over data flow.

Choose the bespoke stack when

- Product or customer data comes from more than Shopify.
- One backend serves several brands, regions or languages.
- Consent and data residency have to be provable, per market.
- Storefronts are built in Webflow or 11ty, or must stay light and React-free.
- AI agents need to check out with mandates and permissions you define.
- SAP or Salesforce is the system of record, and Shopify is one sales channel.
- One sign-in has to cover several stores, or customers pay with Kakao Pay or Samsung Pay.

Hydrogen on top of Xano

The two can be combined. A Hydrogen storefront can read Xano for enriched product data and consent state while Shopify keeps cart and checkout. That keeps one data layer and lets a single client have a React storefront. The consent gap in the server render still applies, so route its calls through the same gate the other storefronts use.

Sources

- Shopify CLI: Hydrogen commands
- Shopify AI Toolkit
- Cloudflare Workers docs
- Google Merchant API
- Salesforce REST API
- SAP Business Accelerator Hub
- Shopify Customer Account API

- Google Identity
- Universal Commerce Protocol
- Agent Payments Protocol (AP2)
- Kakao Pay developers
- Stripe: Korean payment methods
- Shopify agent checkout
- Shopify customer authentication
- Shopify app blocks and theme app extension limits
- Shopify Checkout Kit
- Tauri 2 capabilities
- Hydrogen ModelViewer
- three.js STLExporter
- 3MF specification
- MuleSoft Shopify Connector and its release notes
- Shopify REST Admin API (legacy)
- Shopify REST retirement timeline (Trama)
- Shopify: 2,048 variant limit
- Shopify bulk operations and API limits
- Merchant API product migration guide
- Merchant API versioning and sunset policy and latest updates
- Shopify API versioning
- Shopify: offline token dates (October 2026), access tokens and Dev Dashboard apps
- Google product data specification
- Eight eMudhra CA keys compromised (The Hacker News) and trust stores to patch (Endor Labs)

- RFC 8659: CAA and Set-Cookie attributes (MDN)
- workers-rs and how Workers run (V8 isolates)
- Axum and Tokio
- Cloudflare Containers
- Google Wallet loyalty cards
- Samsung Wallet
- Prototypes: Ondol floor app, Ondol estimate and deposit, fo1 Korea storefront, fo1 portal, CRM Sync desktop, fo1 3D customizer, PIM Sync 3D, STL print-file viewer
- Xano docs
